

Class Activity-2

1. Smart City Air Quality Monitoring

Python code:

```
import pandas as pd

import matplotlib.pyplot as plt

# Read dataset

df = pd.read_csv("air_quality.csv")

plt.figure(figsize=(8,6))

scatter = plt.scatter(

    df['Temperature'],

    df['PM2.5'],

    c=df['NO2'],

    s=df['CO']*40,

    cmap='Reds'

)

plt.xlabel("Temperature")

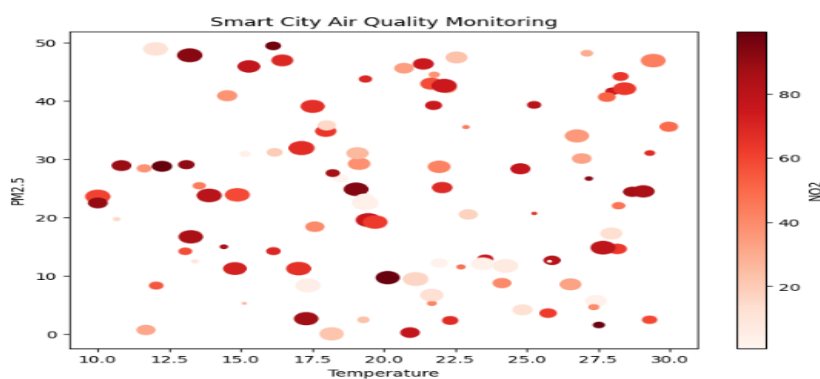
plt.ylabel("PM2.5")

plt.title("Smart City Air Quality Monitoring")

plt.colorbar(scatter, label="NO2")

plt.show()
```

Output:



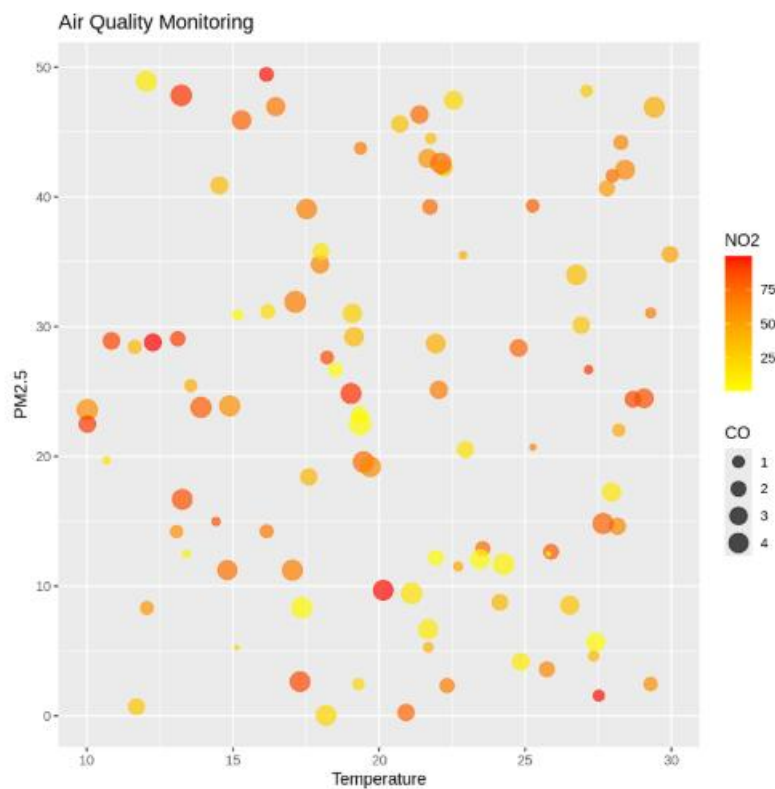
Rcode:

```
library(ggplot2)

data <- read.csv("air_quality.csv")

ggplot(data,
aes(x=Temperature,
    y=PM2.5,
    color=NO2,
    size=CO)) +
geom_point(alpha=0.7) +
scale_color_gradient(low="yellow", high="red") +
labs(title="Air Quality Monitoring")
```

Output:



2. Earthquake Risk Analysis

Python code:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt

# Read dataset
df = pd.read_csv("earthquake.csv")

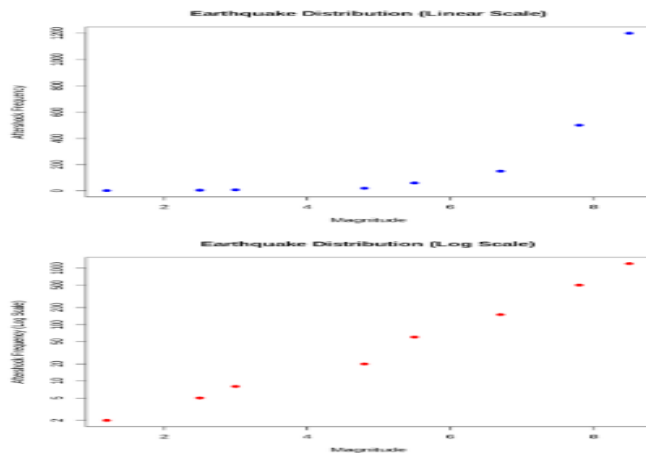
# Linear Scale
plt.figure(figsize=(7,5))
plt.scatter(df["Magnitude"], df["Aftershock_Frequency"],
            color="blue", alpha=0.7)

plt.title("Earthquake Distribution (Linear Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershock Frequency")
plt.grid(True)
plt.show()

# Logarithmic Scale
plt.figure(figsize=(7,5))
plt.scatter(df["Magnitude"], df["Aftershock_Frequency"],
            color="red", alpha=0.7)

plt.yscale("log")
plt.title("Earthquake Distribution (Log Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershock Frequency (Log Scale)")
plt.grid(True)
plt.show()
```

Output:



R code:

```
# Sample Data
```

```
Magnitude <- c(1.2,2.5,3.0,4.8,5.5,6.7,7.8,8.5)
```

```
Aftershock <- c(2,5,8,20,60,150,500,1200)
```

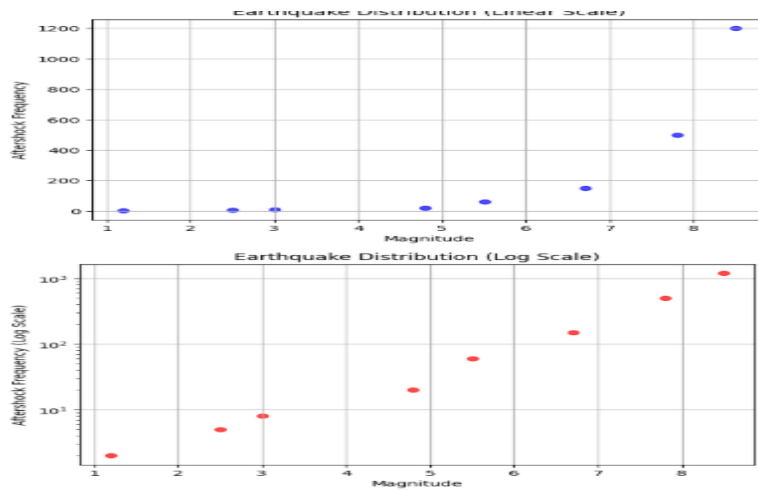
```
# Linear Scale
```

```
plot(Magnitude, Aftershock,
      main="Earthquake Distribution (Linear Scale)",
      xlab="Magnitude",
      ylab="Aftershock Frequency",
      pch=19,
      col="blue")
```

```
# Logarithmic Scale
```

```
plot(Magnitude, Aftershock,
      log="y",
      main="Earthquake Distribution (Log Scale)",
      xlab="Magnitude",
      ylab="Aftershock Frequency (Log Scale)",
      pch=19,
      col="red")
```

Output:



3. Wind Direction Analysis for Airport Operations

Python code:

```
import matplotlib.pyplot as plt
import numpy as np

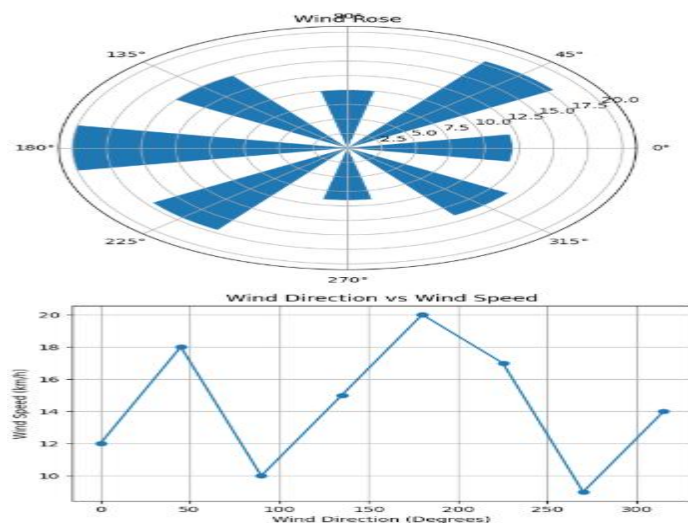
# Sample Data
wind_direction = np.deg2rad([0,45,90,135,180,225,270,315])
wind_speed = [12,18,10,15,20,17,9,14]

plt.figure(figsize=(6,6))
ax = plt.subplot(111, projection='polar')
ax.bar(wind_direction,
       wind_speed,
       width=np.pi/8)
ax.set_title("Wind Rose")
plt.show()

import matplotlib.pyplot as plt
direction = [0,45,90,135,180,225,270,315]
speed = [12,18,10,15,20,17,9,14]
plt.plot(direction, speed, marker='o')
plt.xlabel("Wind Direction (Degrees)")
plt.ylabel("Wind Speed (km/h)")
```

```
plt.title("Wind Direction vs Wind Speed")
plt.grid(True)
plt.show()
```

Output:



R code:

```
# Sample Data
direction <- c(0,45,90,135,180,225,270,315)
speed <- c(12,18,10,15,20,17,9,14)

# Cartesian Plot
plot(direction, speed,
      type="b",
      pch=19,
      col="blue",
      xlab="Wind Direction (Degrees)",
```

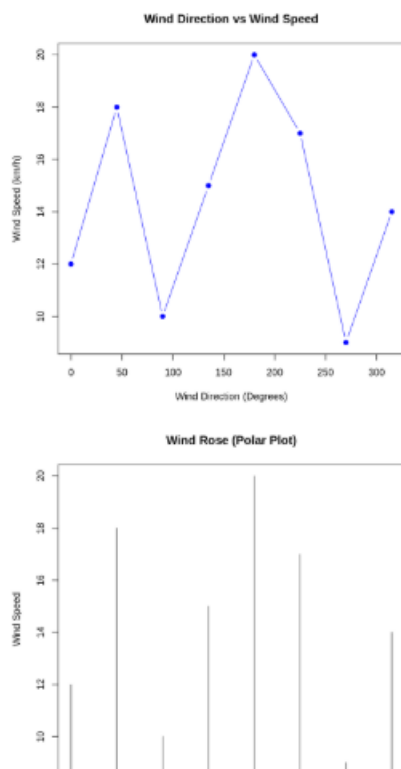
```

ylab="Wind Speed (km/h)",
main="Wind Direction vs Wind Speed")

# Sample Data
direction <- c(0,45,90,135,180,225,270,315)
speed <- c(12,18,10,15,20,17,9,14)
theta <- direction * pi / 180
plot(theta, speed,
      type="h",
      xaxt="n",
      xlab="",
      ylab="Wind Speed",
      main="Wind Rose (Polar Plot)")

```

Output:



4. Hospital Patient Monitoring Dashboard

Python code:

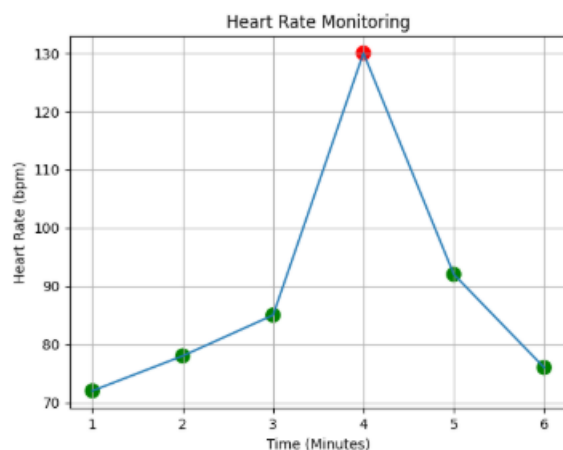
```
import matplotlib.pyplot as plt
```

```

time = [1,2,3,4,5,6]
heart_rate = [72,78,85,130,92,76]
colors = []
for hr in heart_rate:
    if hr < 50 or hr > 120:
        colors.append("red")
    elif hr < 60 or hr > 100:
        colors.append("yellow")
    else:
        colors.append("green")
plt.scatter(time, heart_rate, c=colors, s=100)
plt.plot(time, heart_rate)
plt.xlabel("Time (Minutes)")
plt.ylabel("Heart Rate (bpm)")
plt.title("Heart Rate Monitoring")
plt.grid(True)
plt.show()

```

Output:



R code:

```

time <- c(1,2,3,4,5,6)
heart_rate <- c(72,78,85,130,92,76)
colors <- ifelse(heart_rate > 120 | heart_rate < 50, "red",

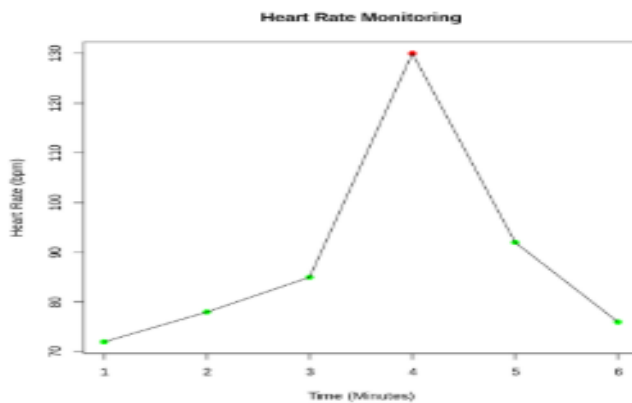
```

```

        ifelse(heart_rate > 100 | heart_rate < 60, "yellow", "green"))
plot(time, heart_rate,
     col=colors,
     pch=19,
     xlab="Time (Minutes)",
     ylab="Heart Rate (bpm)",
     main="Heart Rate Monitoring")
lines(time, heart_rate)

```

Output:



5. Global Climate Change Study

Python code:

```

import numpy as np
import matplotlib.pyplot as plt

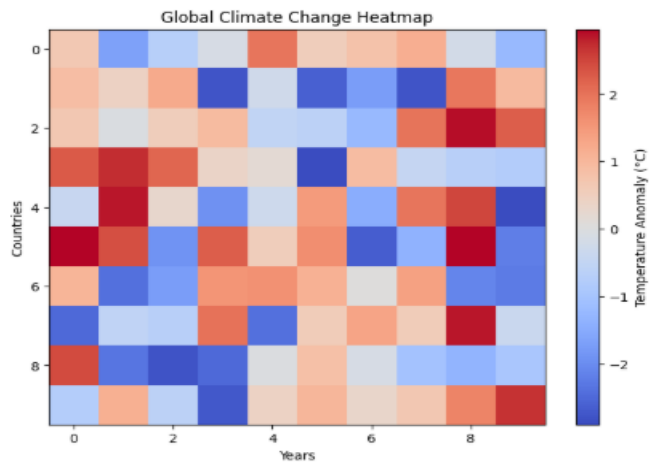
# Sample Data
data = np.random.uniform(-3, 3, (10, 10))

plt.figure(figsize=(8,6))
plt.imshow(data, cmap='coolwarm', aspect='auto')
plt.colorbar(label='Temperature Anomaly (°C)')
plt.title("Global Climate Change Heatmap")
plt.xlabel("Years")
plt.ylabel("Countries")

```

```
plt.show()
```

Output:



R code:

```
# Sample Data
```

```
set.seed(123)
```

```
data <- matrix(runif(100, -3, 3), nrow = 10)
```

```
# Heatmap
```

```
heatmap(data,
```

```
  Colv = NA,
```

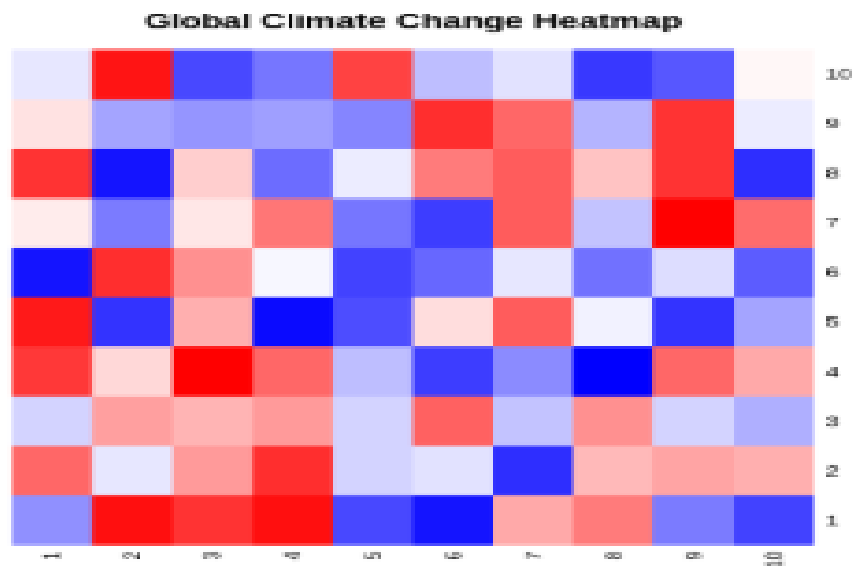
```
  Rowv = NA,
```

```
  scale = "none",
```

```
  col = colorRampPalette(c("blue", "white", "red"))(100),
```

```
  main = "Global Climate Change Heatmap")
```

Output:



6. E-Commerce Sales Analytics

Python code:

```
import matplotlib.pyplot as plt

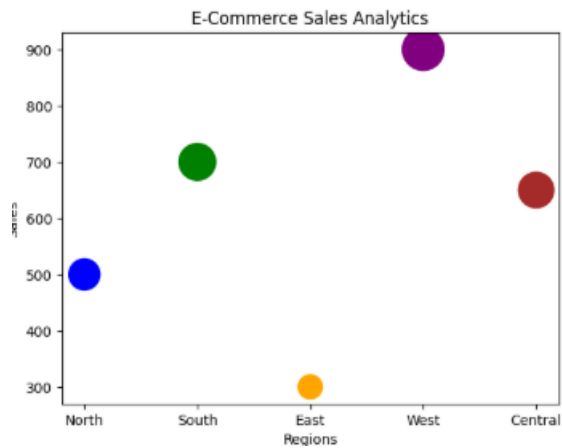
regions = ['North','South','East','West','Central']
sales = [500,700,300,900,650]

categories = ['Electronics','Clothing','Furniture','Grocery','Sports']
colors = ['blue','green','orange','purple','brown']
sizes = [500,700,300,900,650]

plt.scatter(regions, sales,
            c=colors,
            s=sizes)

plt.xlabel("Regions")
plt.ylabel("Sales")
plt.title("E-Commerce Sales Analytics")
plt.show()
```

Output:

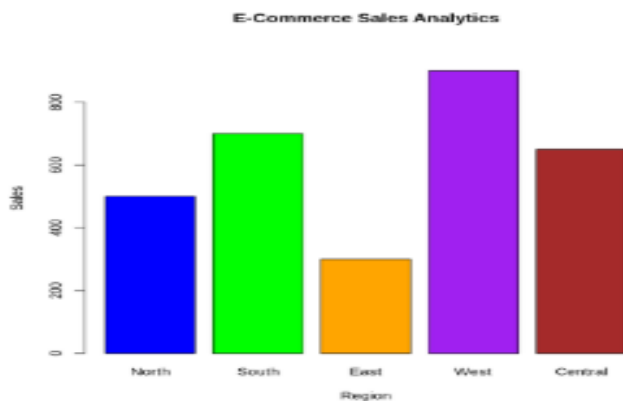


R code:

```
regions <- c("North","South","East","West","Central")
sales <- c(500,700,300,900,650)

# Create a bar plot for categorical regions vs sales
barplot(sales,
        names.arg = regions, # Use regions as labels for the bars
        col = c("blue","green","orange","purple","brown"),
        xlab = "Region",
        ylab = "Sales",
        main = "E-Commerce Sales Analytics",
        ylim = c(0, max(sales) * 1.1) # Set y-axis limits for better visualization
)
```

Output:



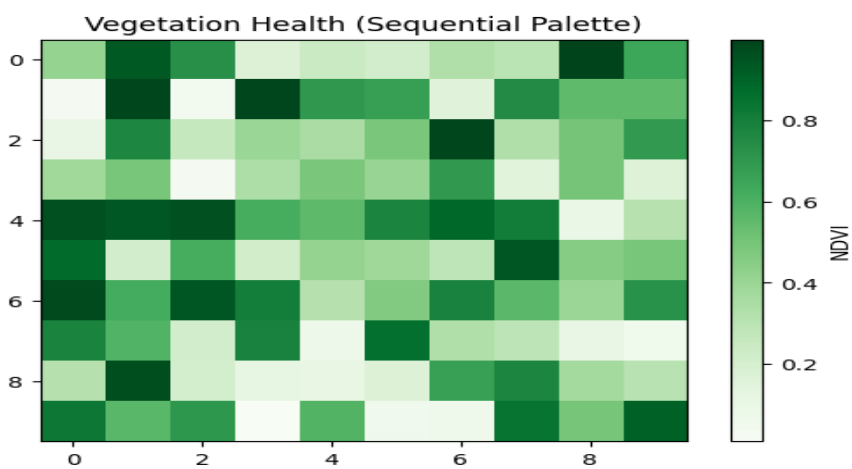
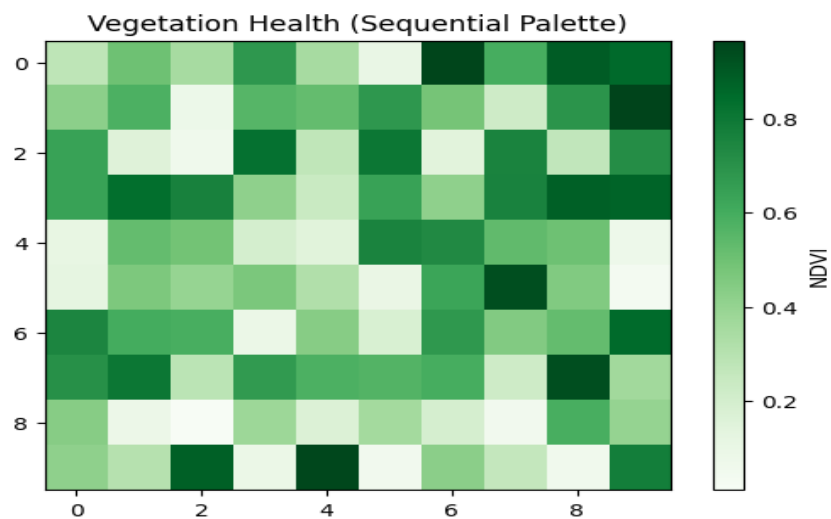
7. Satellite Image Vegetation Analysis

Python code:

```
import numpy as np
import matplotlib.pyplot as plt
ndvi = np.random.uniform(0,1,(10,10))
plt.imshow(ndvi, cmap='Greens')
plt.colorbar(label='NDVI')
plt.title("Vegetation Health (Sequential Palette)")
plt.show()

import numpy as np
import matplotlib.pyplot as plt
ndvi = np.random.uniform(0,1,(10,10))
plt.imshow(ndvi, cmap='Greens')
plt.colorbar(label='NDVI')
plt.title("Vegetation Health (Sequential Palette)")
plt.show()
```

Output:



R code:

```
set.seed(123)

ndvi <- matrix(runif(100,-1,1), nrow=10)

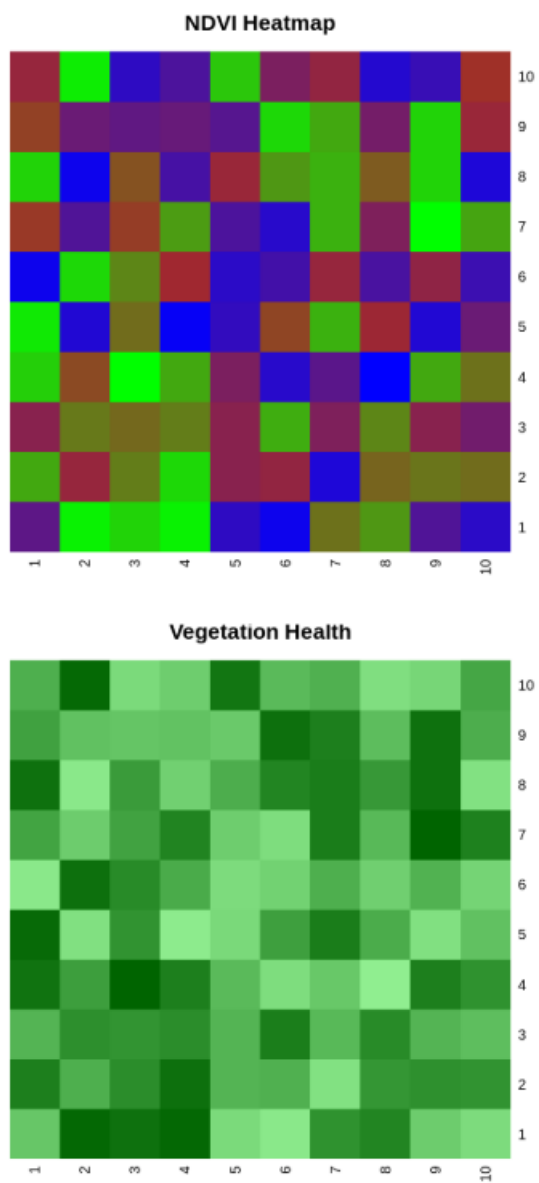
heatmap(ndvi,
        Rowv=NA,
        Colv=NA,
        scale="none",
        col=colorRampPalette(c("blue","brown","green"))(100),
        main="NDVI Heatmap")

set.seed(123)

ndvi <- matrix(runif(100,0,1), nrow=10)
```

```
heatmap(ndvi,
        Rowv=NA,
        Colv=NA,
        scale="none",
        col=colorRampPalette(c("lightgreen","darkgreen"))(100),
        main="Vegetation Health")
```

Output:



8. Cryptocurrency Market Volatility

Python code:

```
import matplotlib.pyplot as plt

days = [1,2,3,4,5]

btc = [30000,32000,35000,34000,36000]

eth = [2000,2100,2300,2200,2400]

sol = [30,35,40,38,42]

ada = [0.5,0.6,0.7,0.65,0.75]

# Linear Plot

plt.figure(figsize=(8,5))

plt.plot(days, btc, label="Bitcoin", color="blue")

plt.plot(days, eth, label="Ethereum", color="green")

plt.plot(days, sol, label="Solana", color="orange")

plt.plot(days, ada, label="Cardano", color="red")

plt.xlabel("Days")

plt.ylabel("Price (USD)")

plt.title("Cryptocurrency Prices (Linear Scale)")

plt.legend()

plt.grid(True)

plt.show()

# Logarithmic Plot

plt.figure(figsize=(8,5))

plt.plot(days, btc, label="Bitcoin", color="blue")

plt.plot(days, eth, label="Ethereum", color="green")

plt.plot(days, sol, label="Solana", color="orange")

plt.plot(days, ada, label="Cardano", color="red")

plt.yscale("log")

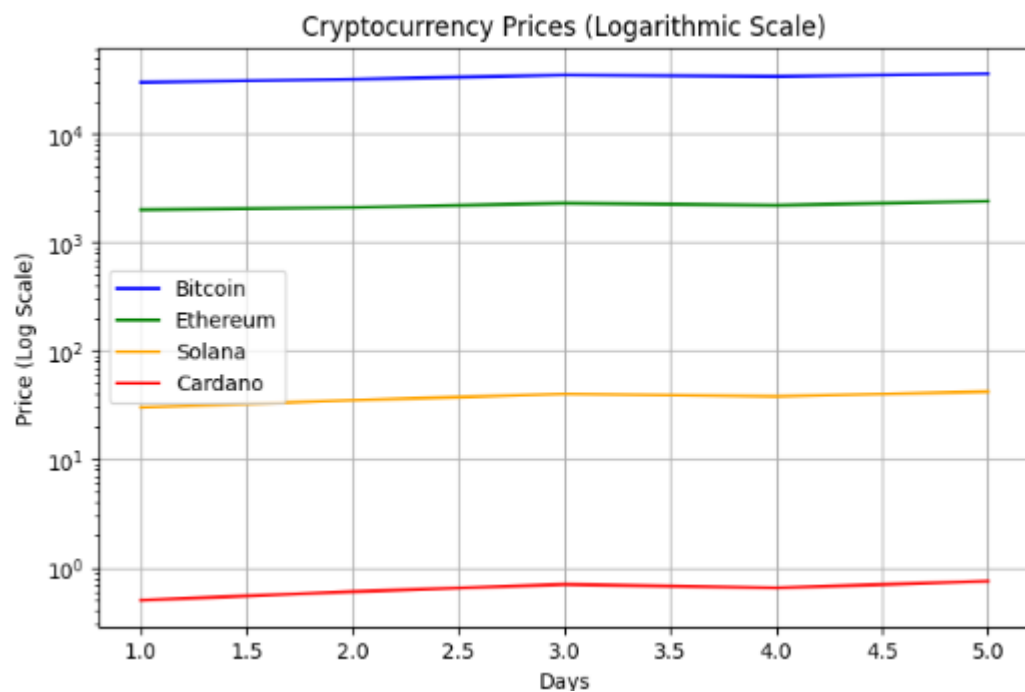
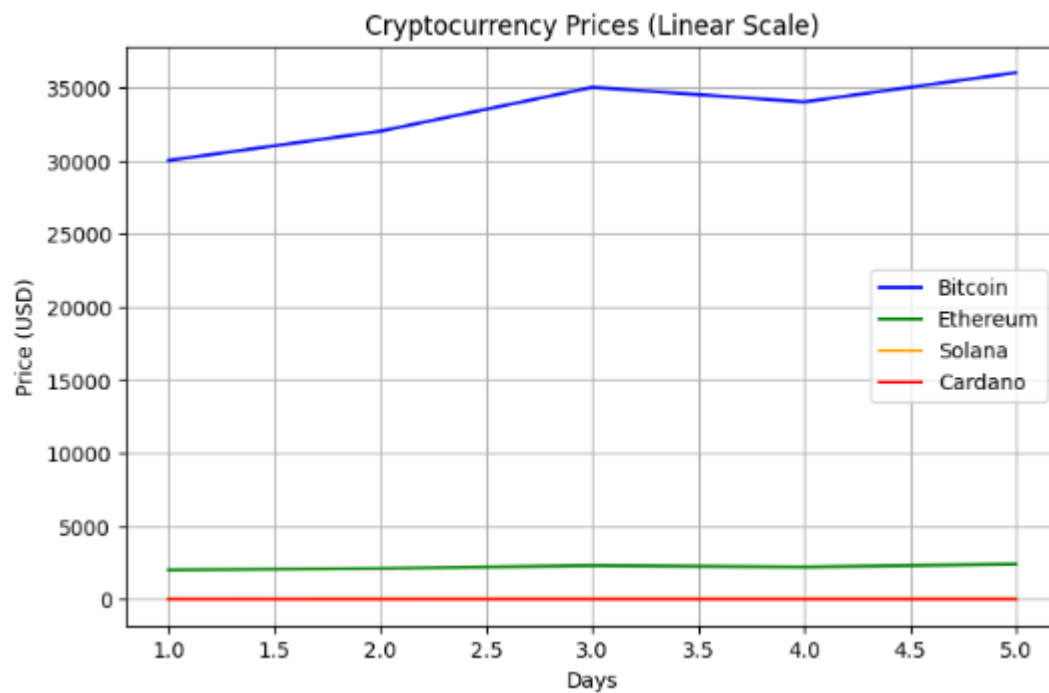
plt.xlabel("Days")

plt.ylabel("Price (Log Scale)")

plt.title("Cryptocurrency Prices (Logarithmic Scale)")
```

```
plt.legend()
plt.grid(True)
plt.show()
```

Output:



R code:

```
days <- c(1,2,3,4,5)
btc <- c(30000,32000,35000,34000,36000)
```

```

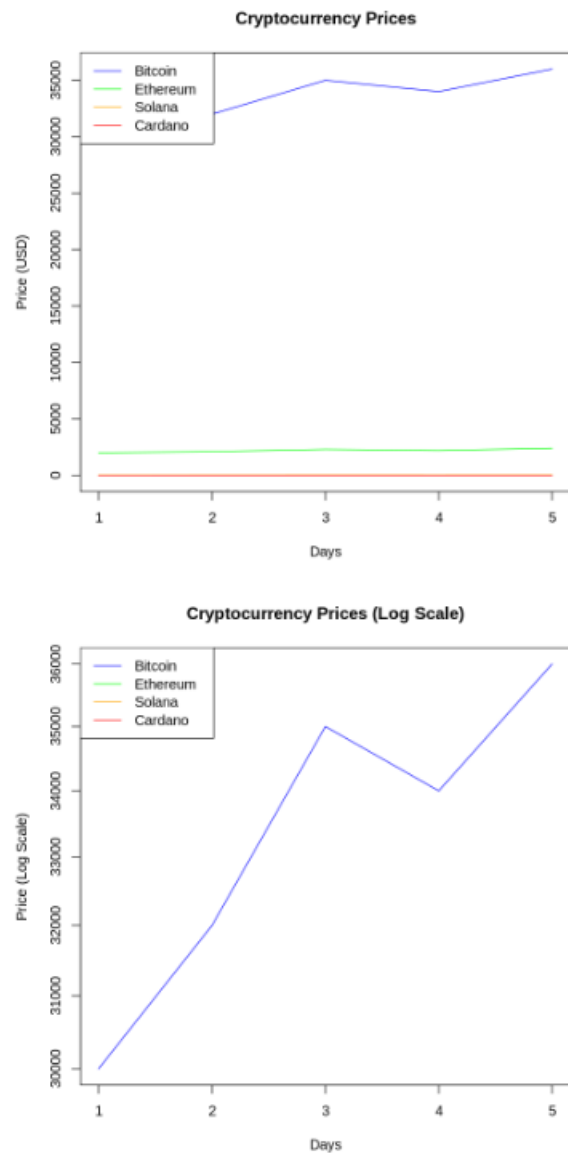
eth <- c(2000,2100,2300,2200,2400)
sol <- c(30,35,40,38,42)
ada <- c(0.5,0.6,0.7,0.65,0.75)

# Linear Plot
plot(days, btc, type="l", col="blue",
      ylim=c(0,36000),
      xlab="Days",
      ylab="Price (USD)",
      main="Cryptocurrency Prices")
lines(days, eth, col="green")
lines(days, sol, col="orange")
lines(days, ada, col="red")
legend("topleft",
      legend=c("Bitcoin","Ethereum","Solana","Cardano"),
      col=c("blue","green","orange","red"),
      lty=1)

# Logarithmic Plot
plot(days, btc, type="l", col="blue",
      log="y",
      xlab="Days",
      ylab="Price (Log Scale)",
      main="Cryptocurrency Prices (Log Scale)")
lines(days, eth, col="green")
lines(days, sol, col="orange")
lines(days, ada, col="red")
legend("topleft",
      legend=c("Bitcoin","Ethereum","Solana","Cardano"),
      col=c("blue","green","orange","red"),
      lty=1)

```

Output:



9. Marathon Runner Performance Analysis

Python code:

```
import matplotlib.pyplot as plt

import numpy as np

# Sample Data

distance = [5,10,15,20,25,30,35,42]

heart_rate = [120,130,138,145,150,155,160,165]

speed = [8,9,10,11,12,11,10,9]
```

```

plt.figure(figsize=(7,5))
scatter = plt.scatter(distance,
                      heart_rate,
                      c=speed,
                      cmap="coolwarm",
                      s=100)
plt.colorbar(scatter,label="Speed (km/h)")
plt.xlabel("Distance (km)")
plt.ylabel("Heart Rate (bpm)")
plt.title("Marathon Runner Performance")
plt.grid(True)
plt.show()

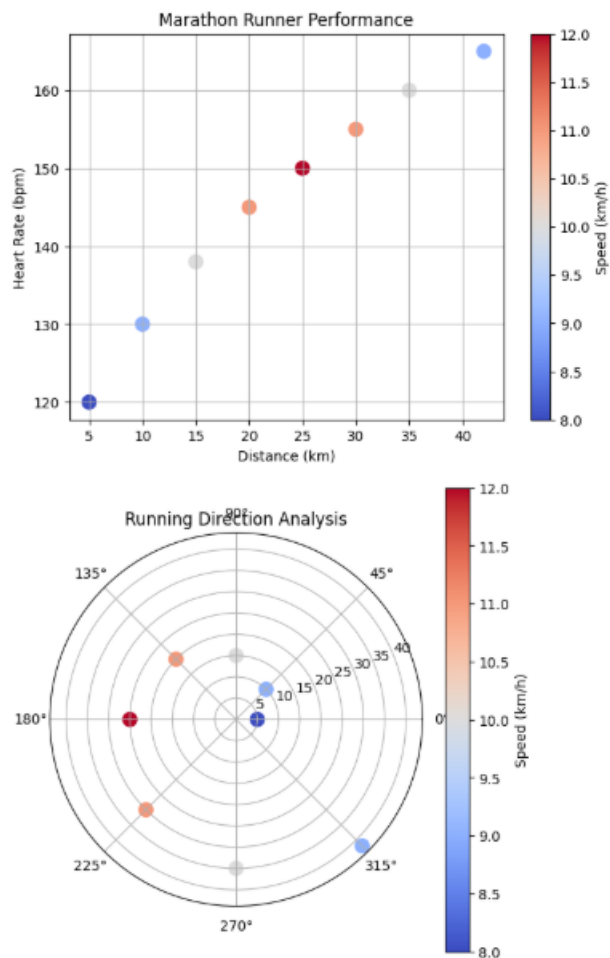
import matplotlib.pyplot as plt
import numpy as np

direction = np.deg2rad([0,45,90,135,180,225,270,315])
distance = [5,10,15,20,25,30,35,42]
speed = [8,9,10,11,12,11,10,9]

plt.figure(figsize=(6,6))
ax = plt.subplot(111, projection="polar")
scatter = ax.scatter(direction,
                    distance,
                    c=speed,
                    cmap="coolwarm",
                    s=100)
plt.colorbar(scatter,label="Speed (km/h)")
plt.title("Running Direction Analysis")
plt.show()

```

Output:



R code:

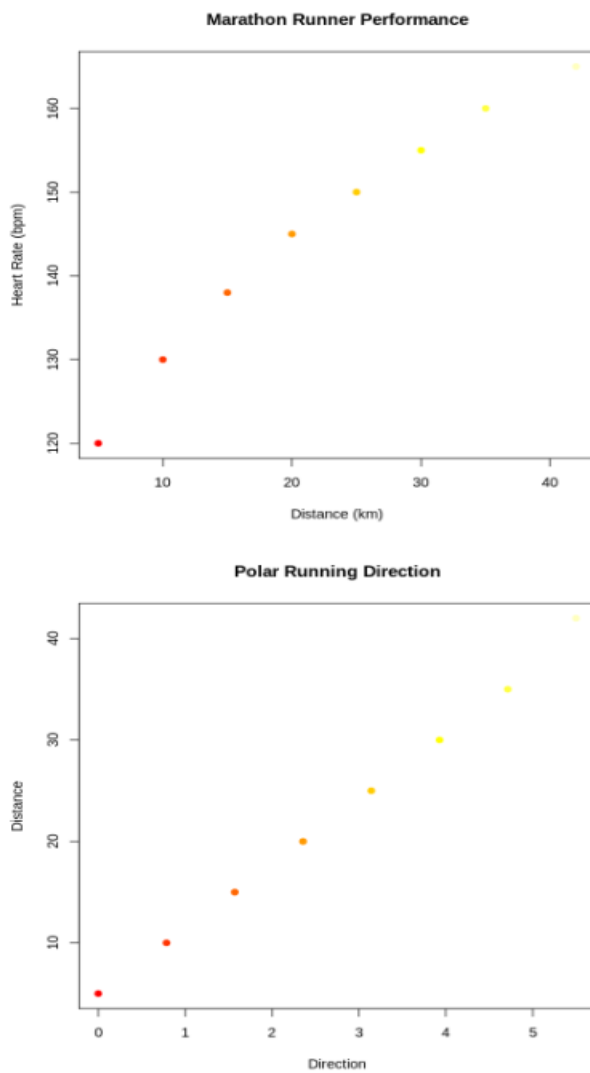
```
distance <- c(5,10,15,20,25,30,35,42)
heart_rate <- c(120,130,138,145,150,155,160,165)
speed <- c(8,9,10,11,12,11,10,9)
colors <- heat.colors(length(speed))
plot(distance,
      heart_rate,
      col=colors,
      pch=19,
      xlab="Distance (km)",
      ylab="Heart Rate (bpm)",
      main="Marathon Runner Performance")
direction <- c(0,45,90,135,180,225,270,315)
distance <- c(5,10,15,20,25,30,35,42)
```

```

theta <- direction * pi / 180
plot(theta,
      distance,
      col=heat.colors(length(distance)),
      pch=19,
      main="Polar Running Direction",
      xlab="Direction",
      ylab="Distance")

```

Output:



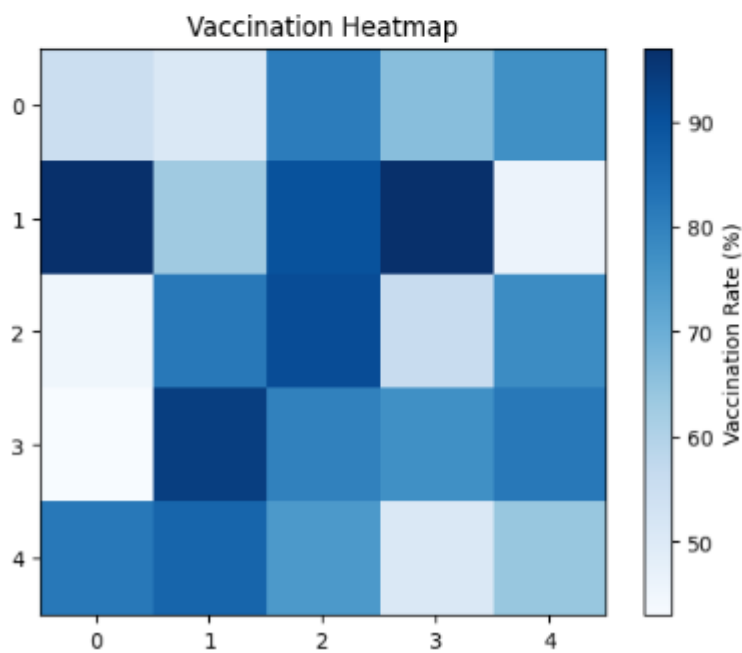
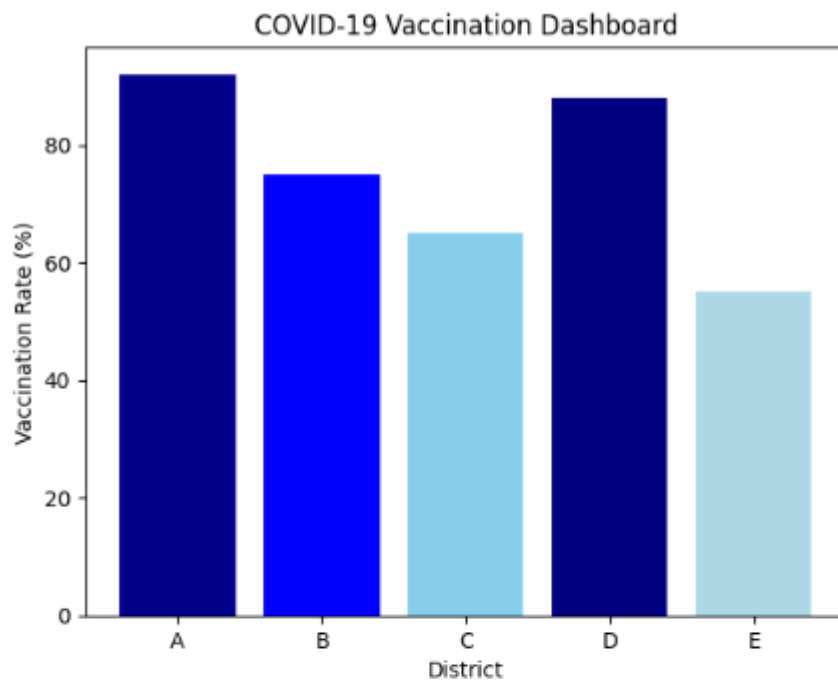
10. COVID-19 Vaccination Campaign Dashboard

Python code:

```
import matplotlib.pyplot as plt
districts = ['A','B','C','D','E']
vaccination = [92,75,65,88,55]
plt.bar(districts,
        vaccination,
        color=['darkblue','blue','skyblue','navy','lightblue'])
plt.xlabel("District")
plt.ylabel("Vaccination Rate (%)")
plt.title("COVID-19 Vaccination Dashboard")
plt.show()

import numpy as np
import matplotlib.pyplot as plt
data = np.random.randint(40,100,(5,5))
plt.imshow(data,cmap="Blues")
plt.colorbar(label="Vaccination Rate (%)")
plt.title("Vaccination Heatmap")
plt.show()
```

Output:



R code:

```
districts <- c("A","B","C","D","E")
```

```
vaccination <- c(92,75,65,88,55)
```

```
barplot(vaccination,
```

```
  names.arg=districts,
```

```
  col=c("darkblue","blue","skyblue","navy","lightblue"),
```

```
  xlab="District",
```

```

ylab="Vaccination Rate (%)",
main="COVID-19 Vaccination Dashboard")
set.seed(123)
data <- matrix(sample(40:100,25), nrow=5)
heatmap(data,
  Rowv=NA,
  Colv=NA,
  scale="none",
  col=colorRampPalette(c("lightblue","blue","darkblue"))(100),
  main="Vaccination Heatmap")

```

Output:

